

Knock Detection

Script: Chapter 12 (capstone: angle-synchronous knock detection)

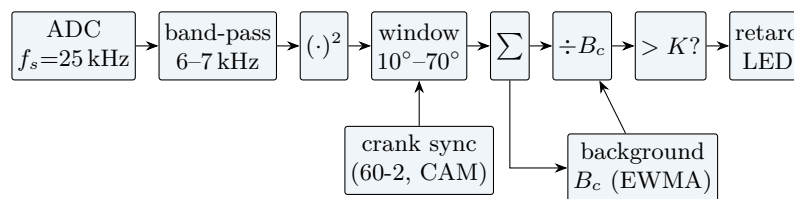
with solutions

Learning goals

- Assemble the methods of the previous sheets into a complete, angle-synchronous knock detector and justify every block quantitatively.
- Evaluate a detector statistically on thousands of labelled combustions: ROC curve, area under the curve, detection rate at a fixed false-alarm rate, choice of the threshold.
- Compare energy-, FFT- and MEM-based knock features and quantify the contribution of band-pass, angle window and background normalisation.
- Run the detector in real time on the ESP32 with crank synchronisation, per-cylinder knock indication and ignition retard; optionally port it to the 8-bit Uno in fixed point.

The signal chain

The ECU does not see the engine. It sees only signals. The ECU never sees the knock – only a vibration signal that also contains noise, the combustion vibration and valve impacts. The standard knock detector of a production ECU is the chain below; every block is one of the previous sheets.



Conventions: $f_s = 25$ kHz (Sheet 10), band-pass = the 10th-order Butterworth biquad cascade of `knock_filter.h` (Sheet 10; group delay 22 samples at 6.5 kHz) or the single biquad `kf_bp2` ($B = 400$ Hz, 20 samples), window 10° – 70° after the TDC of the firing cylinder, feature $E_c = \sum_{\text{window}} y^2[n]$ for cylinder c , normalised $R_c = E_c/B_c$, knock if $R_c > K$. Ground truth: `e.knock_flag[c]` in the C model, the KNK pin of the engine-sim chip in Wokwi. Code: `code/jslinux/sheet12/` (`knockeval.c` plus `knock_filter.h`, `ar.h`, `fft.h` – use your own versions from Sheets 8, 10, 11 if you like) and `code/wokwi/sheet12_knock_esp32/`, `code/wokwi/sheet12_knock_uno/`.

Exercise 12.1 Detector design

PAPER

- Justify every block of the chain with the respective chapter: why sampling at 25 kHz (and what about anti-aliasing), why an angle window and why exactly 10° – 70° ATDC, why a band-pass, why squaring (instead of a linear operation), why summation, why a normalisation and why a threshold. Which blocks are LTI, which are not?
- Compute the window length in samples at 1500, 3000 and 6000 rpm. By how many degrees must the window be shifted to compensate the group delay of the band-pass (22 samples)? Why is a fixed sample shift correct although the window is defined in degrees?
- Energy budget. Show that a knock burst $Ae^{-t/\tau} \sin(2\pi f_1 t)$ that lies completely in the window and is passed with gain 1 contributes $E_k \approx f_s A^2 \tau / 4$ (in V^2 , sum over samples). White noise with standard deviation σ contributes on average $E_n = N_w \sigma^2 \cdot 2B_n / f_s$, where N_w is the window length and B_n the noise bandwidth of the band-pass ($B_n = \int_0^{f_s/2} |H(f)|^2 df = 1161$ Hz for the cascade). Evaluate E_k for $A = 0.2$ V and E_n for $\sigma = 0.05$ and 0.15 V at the three speeds. What do you expect for the detection performance?

- d) Without knock, E_c is approximately $E_n \cdot \chi^2_\nu / \nu$ distributed with $\nu \approx 2B_n T_w$ degrees of freedom (T_w = window duration). Compute ν for the three speeds and, using the 99 % quantiles $\chi^2_{\nu,0.99}/\nu \approx 2.0$ ($\nu = 15.5$), 2.6 ($\nu = 7.7$), 3.4 ($\nu = 3.9$), the threshold for 1 % false alarms in units of the mean noise energy. Why does a fixed threshold K give an rpm-dependent false-alarm rate?
- e) Background normalisation: propose an update rule for B_c that follows slow changes (sensor ageing, rpm, load) but is not pulled up by knocking combustions. Why one background per cylinder?
- f) Control action: on a detected knock the ignition of that cylinder is retarded by Δ_r , without knock it is advanced again by $\Delta_a \ll \Delta_r$ per combustion. Explain the asymmetry and what a false alarm costs.

Solution

a) *Sampling* (Ch. 4/5): the knock modes (6.5/10.5 kHz) need $f_s > 21$ kHz; 25 kHz fits the 50 kHz update of the chip (Sheet 10). A real ECU needs an analog anti-aliasing low-pass below 12.5 kHz, otherwise broadband vibration above $f_s/2$ folds into the knock band. *Crank sync and angle window* (Ch. 1, 4): knock can only occur after the spark, during the combustion of one cylinder, typically 5–30° ATDC, and decays within a few ms; the window excludes the noise of the rest of the cycle and the valve impacts at 90° ATDC (and, in a real engine, injector and piston-slap noise). Since the events are fixed in *angle*, the window must be angle based (time-based windows would move with rpm). 10° start: the combustion vibration and spark noise before; 70° end: before the valve impact. *Band-pass* (Ch. 10): selects the knock resonance and rejects the low-frequency combustion vibration (0.2 V, much larger than light knock) and most of the broadband noise (gain in SNR $\approx (f_s/2)/B_n \approx 11$, i.e. 10 dB). *Squaring* (Ch. 2, nonlinear memoryless system): the knock oscillation has random phase, its linear average is zero; the energy (or the rectified amplitude) is phase independent. *Summation* (Ch. 3, LTI, moving sum): averages the fluctuations of y^2 over the window (matched to the burst duration). *Normalisation*: the absolute energy depends on rpm (window length), sensor sensitivity, mounting position, engine load and ageing; the ratio to the background is dimensionless and comparable. *Threshold*: a binary decision (Ch. 1: from signal to information) with a trade-off between misses and false alarms. LTI: band-pass, summation (with a fixed window). Not LTI: windowing (time-variant, angle-synchronous), squaring (nonlinear), normalisation and threshold (nonlinear, adaptive).

b) $f_s = 25$ kHz, 60° take $60/(6 \cdot \text{rpm})$ s: 6.67 ms = 167 samples at 1500, 3.33 ms = 83 at 3000, 1.67 ms = 42 at 6000 rpm. The delay of $22T_s = 0.88$ ms corresponds to 7.9°, 15.8°, 31.7°. The filter delays the signal by a fixed *time*: the output sample $y[n]$ belongs to the input at $n - 22$; hence the window decision for $y[n]$ must use the crank angle of sample $n - 22$ (in code: a ring buffer of the last angles). Expressed in degrees the shift grows with rpm.

c) $\sum_n y^2[n] \approx f_s \int_0^\infty A^2 e^{-2t/\tau} \sin^2(2\pi f_1 t) dt = f_s A^2 \cdot \frac{\tau}{2} \cdot \frac{1}{2} = f_s A^2 \tau / 4 = 5A^2 \text{ (V}^2\text{)} \Rightarrow E_k = 0.20 \text{ V}^2$ for $A = 0.2 \text{ V}$ (the knock amplitude of the model varies between 0.5 and 1.5 times this value, i.e. $E_k = 0.05 \dots 0.45$, and part of a late burst falls outside the window). Noise: $E_n = N_w \sigma^2 \cdot 2 \cdot 1161/25000$:

rpm	N_w	$E_n, \sigma = 0.05 \text{ V}$	$E_n, \sigma = 0.15 \text{ V}$
1500	167	0.039	0.348
3000	83	0.019	0.174
6000	42	0.010	0.087

At $\sigma = 0.05 \text{ V}$ the knock energy is 5–20 times the mean noise energy: good detection, better at high rpm (shorter window, less noise). At $\sigma = 0.15 \text{ V}$ the ratio is only 0.6–2.3: weak knocks are buried in the noise, the detection rate at low false-alarm rates will be poor, especially at low rpm.

d) $\nu = 2B_n T_w = 15.5, 7.7, 3.9$ at 1500, 3000, 6000 rpm; threshold for 1 % false alarms ≈ 2.0 ,

2.6, $3.4\times$ the mean noise energy. With few degrees of freedom (short window) the noise energy fluctuates more, the χ^2 distribution has a heavier tail, and a higher threshold is needed for the same false-alarm rate. A fixed K is therefore conservative at low rpm and produces more false alarms at high rpm; production ECUs use rpm-dependent thresholds (maps). Note that normalisation removes the *mean* dependence on rpm but not the *shape* of the distribution.

e) Exponentially weighted moving average with clipping: $B_c \leftarrow B_c + \lambda(\min(E_c, 3B_c) - B_c)$, $\lambda \approx 0.05$ (time constant ≈ 20 combustions of that cylinder, i.e. about 1.6 s at 3000 rpm), larger λ during the start (warm-up). The clipping limits the influence of a knocking combustion (alternative: update only with combustions classified as non-knocking – but then a wrong decision can lock the background). One background per cylinder because the transfer path from each cylinder to the sensor differs (distance, block structure): the same knock intensity gives different sensor amplitudes.

f) Knock can damage the engine within few cycles – fast reaction ($\Delta_r \approx 1.5^\circ$ per event). Retard costs efficiency and torque, so the ignition is advanced again slowly ($\Delta_a \approx 0.05^\circ$ per combustion) towards the knock limit; the controller then oscillates closely below the limit. A false alarm costs efficiency (Δ_r that is recovered only after $\Delta_r/\Delta_a = 30$ combustions of that cylinder); a miss risks damage. Hence the threshold is chosen for a low but non-zero false-alarm rate (e.g. 1 %).

Exercise 12.2 Offline evaluation with ground truth

JSLINUX

`knockeval.c` runs the virtual engine at 1500, 3000 and 6000 rpm with sensor noise $\sigma = 0.05$ and 0.15 V (6 scenarios, 1000 evaluated combustions each, knock probability 0.2, knock amplitude $0.2\text{ V} \times (0.5 \dots 1.5)$, light knock) and computes for every combustion eight features, each normalised by its own per-cylinder background:

WB-seg / WB-win	wide-band energy $\sum x^2$ over the whole 180° segment / in the window
BP-seg / BP-win	band-pass (cascade) energy over the segment / in the delay-compensated window
BP2-win	as BP-win with the single biquad ($B = 400\text{ Hz}$)
FFT / FFT-rect	power 6–7 kHz of the zero-padded FFT of the raw window samples, Hann / no window
MEM	power 6–7 kHz of the Burg AR(8) spectrum of the raw window samples

```
gcc -O2 -o knockeval knockeval.c -lm && ./knockeval [knock_amp] [ncomb]
```

- Complete the window logic: integrate x^2 in the window 10° – 70° ATDC and y^2 (cascade, single biquad) while the *delayed* angle is in the window.
- Complete the background update (clipped EWMA, faster during the first 16 combustions of each cylinder).
- Complete the ROC computation in `roc()`: the combustions are sorted by decreasing feature value; every threshold between two values gives a point (FPR, TPR). Compute the AUC (trapezoidal rule) and the detection rate at $\leq 1\%$ false alarms.
- Complete the FFT and MEM features (`fft.h`, `ar.h`).
- Run the program and interpret the AUC and detection tables: what is the contribution of the band-pass, of the angle window, of the delay compensation, of the normalisation (compare the pooled ROC with and without)? Plot `roc.csv`.
- Choose a threshold K for 1% false alarms over all scenarios and compare the per-scenario thresholds with your prediction from 12.1 d).
- Compare the energy, FFT and MEM features. Why is the Hann-windowed FFT so much worse at 1500 rpm than the rectangular one? Which feature would you implement on the microcontroller, and why?

Solution

a)–d) Window logic and background update (code/solutions/jslinux/sheet12/knockeval.c):

```

1      /* background update: knock-robust EWMA (values clipped at
2      3 x background), faster adaptation during the warm-up */
3      for (j = 0; j < NFEAT; j++) {
4          double v = f[j] < 3.0 * bg[cur][j] ? f[j] : 3.0 * bg[cur][j];
5          bg[cur][j] += (nbg[cur] < WARMUP ? LAMBDA_W : LAMBDA) * (v - bg[cur][j]);
6      }
7      nbg[cur]++;
8  }
9      first = (cur < 0);          /* the first segment is incomplete */
10     cur = lc;
11     label = e.knock_flag[cur];
12     nw = 0;
13     for (j = 0; j < NFEAT; j++) acc[j] = 0.0;
14 }
15 d = es_angle_diff(e.theta, es_tdc_deg[cur]);    /* deg ATDC */
16 dh[n & 31] = d;
17 dS = dh[(n - D_SOS) & 31];                      /* angle D samples ago */
18 dB = dh[(n - D_BP2) & 31];
19 n++;
20 acc[0] += x * x;
21 acc[2] += y * y;
22 if (d >= WIN0 && d < WIN1) {
23     acc[1] += x * x;
24     if (nw < MAXW) xw[nw++] = x;
25 }
26 if (dS >= WIN0 && dS < WIN1) acc[3] += y * y;
27 if (dB >= WIN0 && dB < WIN1) acc[4] += y2 * y2;

```

ROC (after sorting by decreasing value; ties are merged):

```

1      for (i = 0; i < m; i++) {
2          double fpr, tpr;
3          if (v[i].l) tp++; else fp++;
4          if (i < m - 1 && v[i + 1].v == v[i].v) continue;    /* ties */
5          fpr = (double)fp / Nn; tpr = (double)tp / P;
6          auc += 0.5 * (tpr + last_tpr) * (fpr - last_fpr);
7          if (fpr <= 0.01) { *tpr1 = tpr; *thr1 = v[i].v; }
8          if (tpr_grid)
9              for (g = 0; g < NGRID; g++) if (fpr <= g / (NGRID - 1.0) && tpr > tpr_grid[g])
10                 tpr_grid[g] = tpr;
11             last_fpr = fpr; last_tpr = tpr;
12     }

```

e) Output (6000 combustions, 1210 with knock = 20.2%):

AUC		WB-seg	WB-win	BP-seg	BP-win	BP2-win	FFT	FFT-rect	MEM
1500	0.05	0.799	0.788	0.957	0.981	0.978	0.804	0.973	0.974
1500	0.15	0.600	0.619	0.738	0.789	0.784	0.620	0.764	0.761
3000	0.05	0.811	0.819	0.985	0.998	0.996	0.955	0.997	0.997
3000	0.15	0.606	0.652	0.747	0.828	0.818	0.717	0.824	0.818
6000	0.05	0.912	0.922	0.988	0.999	0.998	0.997	0.999	0.999
6000	0.15	0.698	0.726	0.822	0.886	0.864	0.826	0.878	0.858
pooled		0.740	0.752	0.897	0.934	0.927	0.846	0.927	0.924
det. @1 % FA									
1500	0.05	20.7	26.4	77.7	82.9	82.4	34.2	80.3	80.8
1500	0.15	3.5	2.0	7.0	20.4	14.4	5.5	15.4	18.9
3000	0.05	29.7	28.2	80.9	95.7	90.0	66.5	92.8	91.9
3000	0.15	2.1	3.7	16.8	30.9	24.6	14.1	27.7	24.6
6000	0.05	52.6	56.5	87.1	96.2	94.3	93.3	97.1	96.7
6000	0.15	4.3	8.7	29.0	37.2	34.8	24.6	31.9	24.6
pooled		11.8	9.3	44.5	59.1	55.9	40.3	57.1	56.4

Band-pass: the decisive block. Without it (WB) the combustion vibration and the noise of all frequencies dominate; AUC 0.74–0.75 pooled, only 9–12 % detection at 1 % false alarms. With it (BP-seg) the AUC rises to 0.90. *Angle window*: BP-win vs. BP-seg: AUC 0.897 → 0.934, detection 44.5 % → 59.1 %; integrating over 180° collects three times more noise energy (and the valve burst, which however is strongly attenuated by the cascade, −44 dB at 8 kHz). For the wide-band features the window helps little because the low-frequency combustion vibration lies inside the window. *Delay compensation*: included in BP-win; without it, at 6000 rpm a 32° portion of the burst would be lost (cf. Sheet 10, 10.3). *Normalisation*: pooled over all scenarios the raw BP-win energy reaches only AUC 0.812 and 9.8 % detection at 1 % false alarms, the normalised ratio 0.934 and 59.1 %: the mean noise energy differs by a factor of 36 between the scenarios (Table in 12.1 c), so no single absolute threshold fits. *Noise*: at $\sigma = 0.15$ V even the best feature detects only 20–37 % of the (light) knocks at 1 % false alarms – consistent with the energy budget of 12.1 c). The ROC curves (`roc.csv`) of BP-win lie far above those of WB-win; at FPR = 5 % the pooled TPR of BP-win is 74 % (WB-win: 28 %).

f) Pooled threshold for 1 % false alarms: $R > 2.00$ (detection 59.1 %) – this is the value $K = 2$ used in 12.3. Per scenario: $R > 1.57, 1.54, 2.02$ ($\sigma = 0.05$ V; 1500/3000/6000 rpm) and 1.79, 2.04, 2.49 ($\sigma = 0.15$ V). The mean ratio of the non-knocking combustions is below 1 (0.54–0.91), because the background contains (clipped) knock energy; in units of the mean noise energy the thresholds are 2.3/2.0 (1500), 2.7/2.4 (3000) and 3.7/3.2 (6000 rpm) – in good agreement with the χ^2 prediction 2.0, 2.6, 3.4 of 12.1 d): the shorter the window, the higher the threshold must be for the same false-alarm rate.

g) The band-pass energy (cascade) is the best and cheapest feature; the single biquad (BP2) is almost as good (AUC 0.927 vs. 0.934): the matched 400-Hz band-pass rejects noise as well as the steep cascade – the Uno version is therefore not much worse. FFT-rect and MEM are comparable to BP-win at $\sigma = 0.05$ V and slightly worse at high noise; they cost much more (FFT of 256 points, or Burg + spectrum evaluation, per combustion) and give no extra information here, since there is only one knock mode in the band. The Hann-windowed FFT is poor at 1500 rpm (AUC 0.80 vs. 0.97): the window is 6.7 ms long, but the burst starts at the beginning of the window (onset 5°–25° ATDC) and has decayed after ≈ 2 ms – exactly where the Hann window is close to zero. At 6000 rpm (window 1.7 ms) the burst fills the window and the difference vanishes. Spectral features become interesting if several knock modes must be weighted, if the knock frequency varies, or to distinguish knock from disturbances at nearby frequencies with very short windows (MEM, Sheet 11). Recommendation for the microcontroller: band-pass energy in the delay-compensated angle window, normalised per cylinder.

Limitation of the model: the valve impacts and the combustion vibration of the virtual engine are deterministic (same amplitude every cycle); a constant disturbance only shifts the feature

and is largely removed by the normalisation. In a real engine these disturbances vary from cycle to cycle and the angle window becomes even more important.

Exercise 12.3 Real-time knock detector

WOKWI

Open `code/wokwi/sheet12_knock_esp32/` (ESP32 + engine-sim chip, 4 knock LEDs, retard LED, logic analyzer). Pins: KNOCK → GPIO34, CRANK → 18, CAM → 19, TDC → 21 (only to check the synchronisation), KNK → 22 (ground truth, only for the evaluation); LEDs cylinder 1–4 on 25, 26, 27, 14; retard PWM on 13. The sketch samples at 25 kHz (deadline loop), filters with `kf_sos` and prints one line `cyl,rpm,R,knock,truth,retard` per combustion and every 200 combustions the confusion counts.

- Crank sync.* Complete the 60-2 decoder in the CRANK interrupt (gap detection: a tooth period more than twice the previous one; revolution from the CAM level at tooth 10) and the angle interpolation `crank_angle()`. The sketch compares the decoded angle at every TDC pulse with 120/300/480/660° and prints the mean error. What limits the accuracy?
- Window.* Complete the integration of y^2 while the delayed angle (ring buffer `th_hist`, `KF_DELAY` = 22 samples) is in the window.
- Decision and action.* Complete `finish_combustion()`: $R = E/B_c$, knock if $R > K$ ($K = 2$ from 12.2), background update, ignition retard (+1.5° per knock, -0.05° per knock-free combustion, max. 9°).
- Run the detector at 1500, 3000, 6000 rpm with knock amplitude 1.0 and 0.2 V and noise 50 and 150 mV (knock probability 20 %). Report detection and false-alarm rates and compare them with 12.2. Watch the LEDs and the retard output.
- Estimate the processing time per sample on the ESP32 and the UART load at 6000 rpm. Why must the sketch not print the samples?
- The engine-sim chip has no spark input: the retard does not influence the knock probability. Describe how a closed knock control loop would behave and how you would test it.

Solution

a) Decoder (`code/solutions/wokwi/sheet12_knock_esp32/sketch.ino`):

```

1 portMUX_TYPE mux = portMUX_INITIALIZER_UNLOCKED;
2 volatile uint32_t t_edge = 0;          /* time of the last rising edge, us */
3 volatile uint32_t t_tooth = 0;         /* duration of one 6 deg tooth, us */
4 volatile int tooth = -1;               /* 0..57, -1: no gap seen yet */
5 volatile int rev = -1;                 /* 0: theta 0..360, 1: 360..720 */
6
7 void IRAM_ATTR on_crank()
8 {
9     uint32_t t = micros();
10    portENTER_CRITICAL_ISR(&mux);
11    uint32_t dt = t - t_edge;
12    t_edge = t;
13    if (t_tooth > 0 && dt > 2 * t_tooth) { /* gap: 18 deg instead of 6 */
14        tooth = 0;
15        if (rev >= 0) rev ^= 1;           /* next revolution */
16    } else {
17        if (tooth >= 0) tooth++;
18        if (tooth > 57) tooth = -1;      /* lost sync */
19        t_tooth = dt;
20    }
21    if (tooth == 10) rev = digitalRead(PIN_CAM) ? 0 : 1; /* phase from CAM
22                                                         (HIGH for 0..180 deg), checked every rev. */
23    portEXIT_CRITICAL_ISR(&mux);
24 }
```

```

25
26 /* crank angle at time t (deg, 0..720) or -1 if not synchronised */
27 static float crank_angle(uint32_t t)
28 {
29     portENTER_CRITICAL(&mux);
30     uint32_t te = t_edge, tt = t_tooth;
31     int k = tooth, r = rev;
32     portEXIT_CRITICAL(&mux);
33     if (k < 0 || r < 0 || tt == 0) return -1.0f;
34     float frac = (float)(int32_t)(t - te) / tt; /* teeth since the last edge */
35     float lim = (k == 57) ? 3.0f : 1.0f; /* tooth 57: 18 deg gap follows */
36     if (frac > lim) frac = lim;
37     if (frac < -1.0f) frac = -1.0f;
38     return 360.0f * r + 6.0f * (k + frac);
39 }

```

After the first gap (at most one revolution) the tooth counter is valid; the revolution flag is toggled at every gap and confirmed by the CAM level at tooth 10 (CAM HIGH for 0°–180°). Mean TDC sync error (PC test of the sketch with the chip model): $\leq 0.01^\circ$ at all speeds – the TDC pulses coincide with the rising edges of teeth 20 and 50, so this check verifies the tooth counting and the phase (a wrong revolution flag would give 360°). Note that the TDC flag must be discarded while the decoder is not yet synchronised (stale pulse). In Wokwi the interrupt latency (a few μs , i.e. $\approx 0.1^\circ$ at 6000rpm) adds to this. Between the teeth the angle is interpolated with the last tooth period; the 1% second-order speed ripple changes the speed by only $\approx 0.2\%$ within one tooth, so the interpolation error stays below 0.01° ; the largest error occurs over the gap (18° extrapolated).

b), c) Window and decision:

```

1  float th = crank_angle(t);
2  th_hist[n_samp & 31] = th;
3  float thd = th_hist[(n_samp - KF_DELAY) & 31]; /* angle of the sample the
4                                                    filter output belongs to */
5  n_samp++;
6  if (th < 0.0f || thd < 0.0f || n_samp < 32) { /* not synchronised */
7      tdc_flag = false;
8      return;
9  }
10
11 /* ground truth: KNK HIGH in 0..90 deg after TDC of the current cylinder */
12 float a_now;
13 int c_now = segment(th, &a_now);
14 if (c_now != last_c_now) { truth[c_now] = false; last_c_now = c_now; }
15 if (a_now < 90.0f && digitalRead(PIN_KNK)) truth[c_now] = true;
16
17 /* crank sync check: decoded angle at the TDC pulse */
18 if (tdc_flag) {
19     float at, thc = crank_angle(t_tdc);
20     tdc_flag = false;
21     if (thc >= 0.0f) {
22         segment(thc, &at);
23         tdc_err_sum += fabsf(at < 90.0f ? at : at - 180.0f);
24         tdc_n++;
25     }
26 }
27
28 /* window on the delayed angle */
29 float a;
30 int c = segment(thd, &a);
31 if (a >= WIN0 && a < WIN1) {
32     if (win_cyl != c) { win_cyl = c; E = 0.0f; }
33     E += y * y;
34 } else if (win_cyl == c && a >= WIN1) {

```

```

35     finish_combustion(c);
36     win_cyl = -1;
37 }

```

```

1 static void finish_combustion(int c)
2 {
3     float R = (nB[c] > 0) ? E / B[c] : 1.0f;
4     bool knock = (nB[c] >= WARMUP) && (R > K_THR);
5     /* background: EWMA of E, clipped at 3 B so that knocking combustions
6        hardly raise it; faster adaptation during the warm-up */
7     if (nB[c] == 0) {
8         B[c] = E;
9     } else {
10        float v = E < 3.0f * B[c] ? E : 3.0f * B[c];
11        B[c] += (nB[c] < WARMUP ? LAMBDA_W : LAMBDA) * (v - B[c]);
12    }
13    nB[c]++;
14    /* ignition retard: fast retard, slow recovery */
15    if (knock) retard[c] = fminf(retard[c] + RET_STEP, RET_MAX);
16    else retard[c] = fmaxf(retard[c] - RET_REC, 0.0f);
17    float rmean = 0.25f * (retard[0] + retard[1] + retard[2] + retard[3]);
18    ledcWrite(PIN_RETARD, (int)(255.0f * rmean / RET_MAX));
19    int led = PIN_LED[cyl_no[c] - 1];
20    if (knock) { digitalWrite(led, HIGH); led_off[c] = millis() + 100; }

```

d) Results (PC test harness running the sketch against the engine-sim chip code; counts after the warm-up; Wokwi results differ in detail):

rpm	knock amp.	noise	combustions	TP	FP	FN	detection	false alarms
3000	1.0 V	50 mV	600	115	1	0	100 %	0.2 %
6000	1.0 V	50 mV	1400	279	4	0	100 %	0.4 %
1500	0.2 V	50 mV	200	32	0	12	72.7 %	0.0 %
3000	0.2 V	50 mV	600	104	1	11	90.4 %	0.2 %
6000	0.2 V	50 mV	1400	267	6	12	95.7 %	0.5 %
1500	0.2 V	150 mV	200	4	1	40	9.1 %	0.6 %
3000	0.2 V	150 mV	600	36	10	79	31.3 %	2.1 %
6000	0.2 V	150 mV	1400	131	30	148	47.0 %	2.7 %

The real-time detector reproduces the offline results of 12.2 (same chain, same K): strong knock is always detected; light knock (0.2 V) at 50 mV noise in 73–96 % of the cases with < 1 % false alarms, and at 150 mV noise the detection collapses while the false-alarm rate rises above the design value at high rpm (fewer degrees of freedom, 12.1 d). The LED of a cylinder flashes for 100 ms at each detection, the retard LED brightness follows the mean retard (sawtooth: fast up, slow down).

e) Per sample: `analogRead`, 5 biquads (25 multiply-adds in float), angle interpolation (one division), a few comparisons – some hundred instructions, i.e. $\approx 1\text{--}2\ \mu\text{s}$ at 240 MHz plus the ADC conversion, well below $40\ \mu\text{s}$ (Wokwi timing is only indicative). UART: at 6000 rpm 200 combustions/s $\times \approx 22$ characters = 4400 characters/s, 38 % of the 11 520 characters/s at 115 200 baud; the 4-kB TX buffer decouples the printing from the sampling loop. Printing the raw samples ($25\ 000/\text{s} \times \geq 5$ characters) would exceed the UART capacity tenfold.

f) In a closed loop the retard lowers the cylinder pressure and temperature and thus the knock probability; the controller then settles into a limit cycle around the knock limit (retard steps after each knock, slow advance in between). Testing requires a knock model that depends on the spark advance, e.g. $p_{\text{knock}} = f(\text{spark_adv} - \text{retard})$, i.e. a SPARK input or a serial/I²C command input of the chip. One would then record retard and knock rate per cylinder over time and check stability (no persistent heavy knock, small mean retard) for different noise levels and false-alarm rates.

Exercise 12.4 Port to the Arduino Uno**CHALLENGE**

Port the detector to the Uno (`code/wokwi/sheet12_knock_uno/`, starting from the Q15 biquad of Sheet 10): Timer1 ISR at 25 kHz, single biquad `kf_bp2_q14`, window from the TDC pulse (period in samples of the previous combustion, cylinder 1 = TDC while CAM is HIGH), energy $\sum(y^2 \gg 8)$ in 32 bit, integer background $B += (\min(E, 3B) - B) \gg 4$, decision $E > 2B$ without division. Report detection and false-alarm rates as in 12.3 and the limits of the port.

Solution

See `code/solutions/wokwi/sheet12_knock_uno/sketch.ino`. The ISR adds only the window bookkeeping and the KNK check (`PIND`) to the ISR of Sheet 10 (estimated ≈ 500 of 640 cycles per sample); the decision, background, LEDs and printing run in `loop()` once per combustion. Results of the PC test (knock probability 20 %): 1.0 V/50 mV at 3000 rpm: 107 TP, 5 FP, 0 FN, 488 TN; 0.2 V/50 mV: 87 % detection / 0 % false alarms at 1500 (34/39), 92.5 % / 1.2 % at 3000 (99/107), 91.9 % / 0.1 % at 6000 rpm (260/283); 0.2 V/150 mV at 3000: 36 % / 3.4 %. That is about the performance of the ESP32 – consistent with 12.2, where the single biquad (BP2-win) was almost as good as the cascade. Limits: CPU load $\approx 75\text{--}80\%$ (no room for a second filter, FFT or MEM), no 60-2 decoder (the TDC pulse is a ground-truth helper of the simulator – a real implementation must decode the crank wheel, e.g. with the input capture unit of Timer1, which competes with the sampling interrupt), window positions from the previous combustion (lag at fast rpm changes), 10-bit ADC (resolution $4.9\text{ mV}/0.6 \approx 8\text{ mV}$ at the sensor, sufficient for knock amplitudes $\gg$ noise), and no floating point for diagnostics.

Deliverables

12.1 on paper (block justification, numbers for b)–d)). `knockeval`: AUC and detection tables, pooled ROC plot (BP-win, WB-win, FFT, MEM), chosen threshold with justification. Wokwi: the confusion counts for the settings of 12.3 d) and a screenshot of the LEDs; a short discussion (max. one page) of the remaining weaknesses of your detector.